

Recovering the Hidden Digit

An electrical tomography inverse problem

Travis Whitney Cole Seifert Alexander Nutt

AI 539, Oregon State University

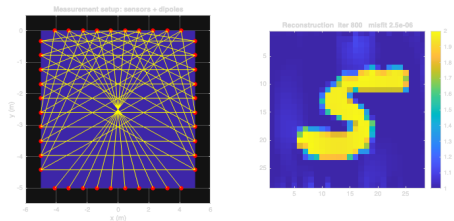
Project 1 Competition, Spring 2026

What we are solving

Electrodes around the edge of a box push small currents through it and measure the resulting voltages. We are handed **1900 of these readings**.

Hidden inside the box is a **handwritten digit**, drawn as a region that conducts electricity differently from its surroundings. Our job is to figure out which digit is inside, using only the readings.

Going **forward** (digit to readings) is a quick physics simulation we call F . We have to go **backward** (readings to digit), which is the hard part.



The sensor setup (left) and a reconstruction we produced (right).

Why it is hard

There are two problems, and together they make a direct approach fail.

- 1. Many digits give nearly the same readings.** The edge sensors only pick up a blurred summary of the inside, so very different digits can produce almost identical readings. Think of a shadow on a wall: many shapes cast the same shadow, so the shadow alone does not tell you the shape.
- 2. Small measurement errors blow up.** The readings are slightly noisy, and undoing the blur magnifies that noise a lot.

So “find any image whose readings match” gives garbage. We have to tell the method what a real answer should look like. That piece of knowledge is called a **prior**, and choosing it well is the whole game.

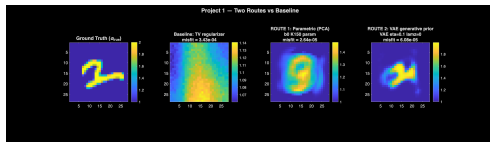
What we tried, and the lesson it taught us

We tried priors from simple to smart:

- ▶ **Generic rules** (smooth, or mostly blank): only blurry blobs.
- ▶ **Linear prior from MNIST (PCA)**: clean digit shape and the *lowest* error, but the **wrong digit**.
- ▶ **Nonlinear prior** (small neural net): the **correct digit**.

The surprise: the lowest-error method gave the wrong answer. Matching the readings better than the noise allows just fits noise.

The prior matters, not the error.



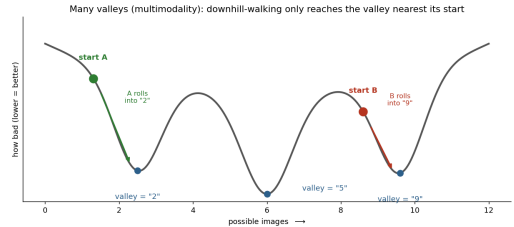
True digit, then three methods. The lowest-error result (third panel) is the wrong digit.

The real obstacle: many answers fit

Because several digits each fit the readings reasonably well, the landscape of possible answers has **many separate low points**, one near each plausible digit.

A method that just walks downhill settles into whichever low point it starts nearest, and stops there. We checked this directly: start the search at a 2 and it returns a 2; start it at a 5 and it returns a 5. The readings alone do not choose the digit.

The fix is a search that looks at **all** the options at once, instead of rolling into the nearest one.



Downhill search reaches only the nearest "valley," so it can lock onto the wrong digit.

Our method

The insight. The project says the hidden image is essentially one of the MNIST digits, and we have all 60,000. So we check them all instead of searching blindly.

Stage 1, identify. For each of the 60,000 real digits, simulate its readings with F and compare to the measured ones. Keep the closest. Checking every option means we cannot lock onto the wrong digit.

Stage 2, polish. From that best match, take a few downhill steps to sharpen the exact shape. Safe now, since Stage 1 found the right digit.

Template matching: simulate each of 60,000 real digits, keep the closest match



The method, in math

Let $\mathcal{D} = \{m_1, \dots, m_N\}$ be the $N = 60,000$ MNIST training images, $\sigma_b = 1$ the background, F the ERT forward map, and y_{obs} the measurements. Define the data misfit

$$L(x) = \frac{1}{2} \|F(\sigma_b \mathbf{1} + x) - y_{\text{obs}}\|_2^2.$$

Stage 1 (template matching).

$$i^* = \arg \min_{i \in \{1, \dots, N\}} L(m_i), \quad x^{(0)} = m_{i^*}.$$

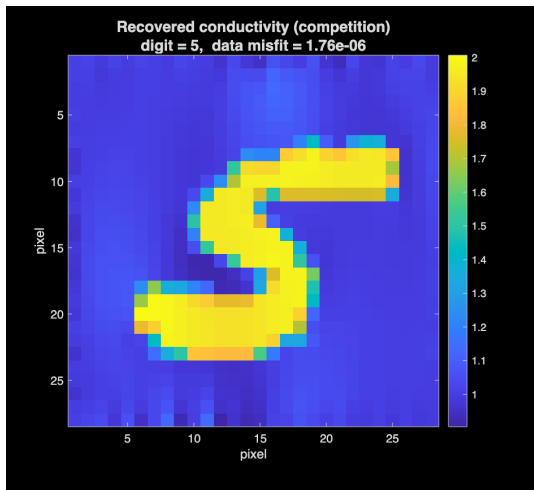
Stage 2 (warm-started refinement). With $\theta^{(0)} = \mathbf{0}$, for $k = 0, 1, \dots, K - 1$:

$$\begin{aligned} r^{(k)} &= F(\sigma_b \mathbf{1} + x^{(k)}) - y_{\text{obs}}, \\ \theta^{(k+1)} &= \gamma \theta^{(k)} + \eta J(x^{(k)})^\top r^{(k)}, \\ x^{(k+1)} &= \max(x^{(k)} - \theta^{(k+1)}, -\sigma_b + \varepsilon). \end{aligned}$$

Return $\hat{\sigma} = \sigma_b \mathbf{1} + x^{(K)}$.

No explicit regularizer in L . The prior is the discrete dictionary $\mathcal{D}$ itself.

What we recovered



From the competition data, the hidden digit is a **5**.

The eight best matches were almost all the same 5, so we are confident in the identity.

After polishing, the reconstruction matches the readings about **200 times better** than the starter method, and the picture is clean and sharp.

Summary

- ▶ The problem is hard because many digits fit the readings and noise blows up, so matching the readings is not enough.
- ▶ Across everything we tried, the prior mattered more than the error. The lowest-error method actually returned the wrong digit.
- ▶ Our method uses the strongest prior available, the real digits themselves, and a search that checks every one, so it never gets trapped on the wrong digit. Then it polishes the winner.
- ▶ **Result: a clean 5**, recovered far more accurately than the baseline.

Thank you. Questions?

Same method on 10 unseen digits: 9 of 10 correct



Labels read **true** → **recovered**. The one miss (5 → 6) is a handwritten 5 with a looped bottom that resembles a 6, and it was the least confident match. The competition data is not a close call: the top eight matches were almost all 5's.